

Stage 1: Foundations

Sigmoid

$\sigma(x) = 1 / (1 + \exp(-x))$

```
def sigmoid(z: float) -> float:
    return 1 / (1 + math.exp(-z))
```

Python

SoftMax

```
def softmax(scores: np.ndarray) -> list[float]:
    scores = scores - np.max(scores, axis=-1, keepdims=True)
    return np.exp(scores) / np.sum(np.exp(scores), axis=-1, keepdims=True)
```

Python

CrossEntropy

CE Including log_softmax

```
def cross_entropy(logits, labels):
    # logits: (N, C)
    # labels: (N,) class ids or (N, C) one-hot
    shifted = logits - np.max(logits, axis=-1, keepdims=True)
    log_probs = shifted - np.log(np.sum(np.exp(shifted), axis=-1, keepdims=True))
    if labels.ndim == 1:
        loss = -log_probs[np.arange(len(labels)), labels] # (B, H)
    else:
        loss = -np.sum(labels * log_probs, axis=-1)
    return loss.mean()
```

Python

CE with Mask (seq_len,)

```
def masked_ce_loss(logits, targets, mask):
    shifted = logits - np.max(logits, axis=-1, keepdims=True)
    log_probs = shifted - np.log(np.sum(np.exp(shifted), axis=-1, keepdims=True))
    loss = -log_probs[np.arange(len(targets)), targets]
    denom = mask.sum()
    if denom == 0:
        return 0.0
    return (loss * mask).sum() / denom
```

Python

Binary Cross Entropy

```
def binary_cross_entropy(y_true, y_pred, epsilon: float = 1e-15) -> float:
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    loss = - (np.log(y_pred) * y_true + np.log(1 - y_pred) * (1 - y_true))
    return loss.mean()
```

Python

LayerNorm

```
def layer_norm(x, gamma, beta, eps=1e-5):
    mean = np.mean(x, axis=-1, keepdims=True)
    var = np.var(x, axis=-1, keepdims=True)
    out = (x - mean) / np.sqrt(var + eps)
    return gamma * out + beta
```

Python

RMSNorm

```
def rmsnorm(x: np.ndarray, g: np.ndarray, eps: float = 1e-5) -> np.ndarray:
    """
    RMSNorm(x) =  $\frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}}$   $\odot \gamma$ 
    x : np.ndarray of shape (batch_size, features)
    g : np.ndarray of shape (features,) - gain parameter
    """
    x = x / (np.linalg.norm(x, axis=-1, keepdims=True) + eps)
    x = x / (np.sqrt(np.mean(x ** 2, axis=-1, keepdims=True) + eps)) * g[None, :]
    return x
```

Python

Dropout

```
class DropoutLayer:
    def __init__(self, p: float):
        # self.p: the dropout rate
        self.p = p
        self.mask = None

    def forward(self, x: np.ndarray, training: bool = True) -> np.ndarray:
        if not training or self.p == 0:
            return x
        mask = np.random.random(x.shape) < 1 - self.p
        self.mask = mask
        x = x * self.mask / (1 - self.p) # Scale up
        return x

    def backward(self, grad: np.ndarray) -> np.ndarray:
        if self.mask is None:
            return grad
        return (grad * self.mask) / (1 - self.p)
```

Python

Cosine similarity

Two Vector

```
v1 = v1 / np.linalg.norm(v1, axis=-1)
v2 = v2 / np.linalg.norm(v2, axis=-1)
return np.sum(v1 * v2)
```

Python

One Matrix

```
X = np.array(X)
X = X / (np.linalg.norm(X, axis=-1, keepdims=True) + 1e-7)
S = X @ X.T
```

Python

Binary Logistic Regression

```
def logistic_regression_gradient_descent(X, y, alpha, iterations):
    B, d = X.shape
    std = np.sqrt(2.0 / d)
    theta = np.random.normal(0.0, std, size=d) # (d,) Xavier init

    for _ in range(iterations):
        logits = X @ theta # (B, )
        y_hat = 1 / (1 + np.exp(-logits))
        gradient = (X.T @ (y_hat - y)) / B
        theta -= alpha * gradient

    return theta
```

Python

Linear Regression Training

```
def linear_regression_gradient_descent(X: np.ndarray, y: np.ndarray,
alpha: float, iterations: int) -> np.ndarray:
    B, d = X.shape # (B, D)
    std = np.sqrt(2.0 / d)
    theta = np.random.normal(0.0, std, size=d) # (D,) Xavier init

    for _ in range(iterations):
        y_hat = X @ theta # (B, )
        error = y_hat - y # (B, ) Use MSE
        gradient = (X.T @ error) / B # (D, 1)
        theta -= alpha * gradient

    return theta
```

Python

Multi-Class Classification

```
def multiclass_logistic_regression_gradient_descent(X, y, alpha,
iterations, num_classes):
    B, d = X.shape
    std = np.sqrt(2.0 / (d + num_classes))
    W = np.random.normal(0.0, std, size=(d, num_classes)) # Xavier init

    for _ in range(iterations):
        logits = X @ W # (B, C)
        logits -= logits.max(axis=1, keepdims=True)
        probs = np.exp(logits)
        probs /= probs.sum(axis=1, keepdims=True) # (B, C)

        y_onehot = np.eye(num_classes)[y] # (B, C)
        gradient = (X.T @ (probs - y_onehot)) / B # (D, C)
        W -= alpha * gradient

    return W
```

Python

K-Means

```
def k_means_np(points, k, initial_centroids, max_iterations=100):
    X = np.asarray(points, dtype=float) # (N, D)
    centroids = np.asarray(initial_centroids, dtype=float) # (K, D)

    for _ in range(max_iterations):
        distances = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2) # (N, K)
        labels = distances.argmin(axis=1) # (N,)
        new_centroids = centroids.copy()
        for i in range(k):
            mask = labels == i
            if mask.any():
                new_centroids[i] = X[mask].mean(axis=0)
            if np.linalg.norm(new_centroids - centroids) < 1e-6:
                break
        centroids = new_centroids

    return centroids
```

Python

Stage 2: Metrics & Losses

```
def calculate_f1_score(y_true, y_pred):
    eps = 1e-7
    TP = np.sum((y_true == 1) & (y_pred == 1))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    precision = TP / (TP + FP + eps)
    recall = TP / (TP + FN + eps)
    f1 = 2 * precision * recall / (precision + recall + eps)
```

Python

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

TopK retrieval

```
def top_k_accuracy(y_scores: np.ndarray, y_true: np.ndarray, k: int) -> float:
    y_s = np.argsort(-y_scores, axis=-1)[:, :k] # (B, k)
    res = (y_s == y_true[:, None]).any(axis=-1).astype(float)
    return np.mean(res)
```

Python

Triplet loss - L2 distance

```
def triplet_margin_loss(anchor: np.ndarray, positive: np.ndarray, negative: np.ndarray,
margin: float = 1.0) -> float:
    a_p_dis = np.sqrt(((anchor - positive) ** 2).sum(axis=-1, keepdims=False))
    a_n_dis = np.sqrt(((anchor - negative) ** 2).sum(axis=-1, keepdims=False))
    loss = np.maximum(a_p_dis - a_n_dis + margin, 0.0)
    return float(np.mean(loss))
```

Python

InfoNCE_Loss (A, B Version)

```
def infoNCE_loss(A: np.ndarray, B: np.ndarray, temperature: float) -> float:
    """
    A: Query embeddings of shape (N, d)
    B: Key embeddings of shape (N, d)
    temperature: Temperature scaling parameter (tau > 0)
    """
    # Normalize embeddings
    A = A / np.linalg.norm(A, axis=-1, keepdims=True)
    B = B / np.linalg.norm(B, axis=-1, keepdims=True)

    # Similarity matrix: (N, N)
    logits = A @ B.transpose(1, 0) / temperature
    labels = np.arange(logits.shape[0])

    # Cross-entropy loss
    def cross_entropy(logits, labels):
        logits = logits - np.max(logits, axis=-1, keepdims=True) # numerical stability
        log_probs = logits - np.log(np.sum(np.exp(logits), axis=-1, keepdims=True))
        return -np.mean(log_probs[np.arange(logits.shape[0]), labels])
    loss = cross_entropy(logits, labels)
    return loss
```

Python

MSE / MAE

```
mae_loss = np.mean(np.abs(y_true - y_pred))
rmse_loss = np.sqrt(np.mean((y_true - y_pred) ** 2))
```

Python

KL divergence

KL(P || Q) = 用Q 拟合P 的代价

```
kl = np.sum(p * (np.log(p) - np.log(q)), axis=-1)
```

Python

GRPO KL Estimator, It is too expensive to enumerate the whole distribution.

```
def kl_divergence_estimator(pi_theta: np.ndarray, pi_ref: np.ndarray) -> np.ndarray:
    """
    Formula: D_KL = (pi_ref / pi_theta) - log(pi_ref / pi_theta) - 1
    pi_theta: Current policy probabilities for each sample
    pi_ref: Reference policy probabilities for each sample
    """
    r = pi_ref / pi_theta
    return r - np.log(r) - 1
```

Python

- Its expectation equals $KL(pi_theta || pi_ref)$.
- It is non-negative per token, so it is more stable than $\log(pi_theta / pi_ref)$.

Label smoothing

```
def smooth_labels(y_true, num_classes, epsilon):
    """
    y_true: Iterable[int] of shape (N,) with values in [0, K-1]
    num_classes: int, total number of classes (K)
    epsilon: float in [0, 1]
    """
    K = num_classes
    N = y_true.shape[0]
    res = np.full(
        shape=(N, K),
        fill_value=epsilon / K
    )
    res[np.arange(N), y_true] = 1 - epsilon + epsilon / K
    return res
```

Python

Perplexity

```
def calculate_perplexity(probabilities: list[float]) -> float
    return np.exp(np.mean(-np.log(probabilities)))
```

Python

$$PPL = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})\right)$$

Stage 3: Neural Network Components

BPE Tokenizer

- Count all adjacent token pairs across the corpus, weighted by word frequency
- Find the most frequent pair
- Merge that pair everywhere it appears in the corpus (replacing the two tokens with their concatenation)
- Record the merge operation
- Repeat for the specified number of merges

Attentions

Causal Self-Attention

Generate attention mask **note this is the reversed mask, True means disposal**

```
mask = np.arange(L) [None, :] > np.arange(L)[:, None]
# or
causal_mask = np.triu(np.ones((L, L)), k=1).astype(bool)
```

Python

```
d_k = 0. shape[-1]
score = Q @ K.transpose(0, 2, 1) / np.sqrt(d_k)
mask = np.triu(np.ones_like(score), k=1)
score = np.where(mask, -np.inf, score)
score = score - np.max(score, axis=-1, keepdims=True)
score = np.exp(score) / np.sum(np.exp(score), axis=-1, keepdims=True)
return score @ V
```

Python

SwiGLU (Modern)

```
def swiglu_ffn(x, W_gate, W_up, W_down):
    gate = x @ W_gate
    up = x @ W_up
    hidden = silu(gate) * up
    output = hidden @ W_down
    return output
```

Python

SwiGLU 通常在同等参数/计算量下经验效果略好，而且 SiLU 形式更简单、梯度更平滑，所以 LLaMA 等模型采用了它。

$FFN(x) = W_{down}(SiLU(W_{gate} x) * (W_{up} x))$

Multi-Head Attention

```
def multi_head_attention(Q: np.ndarray, K: np.ndarray, V: np.ndarray, n_heads: int):
    d_model = Q.shape[-1]
    seq_len = Q.shape[0]

    d_q, remain = divmod(d_model, n_heads)
    if remain != 0:
        raise ValueError("Wrong n_head")

    Q = Q.reshape(seq_len, n_heads, d_q).transpose(1, 0, 2)
    K = K.reshape(seq_len, n_heads, d_q).transpose(1, 0, 2)
    V = V.reshape(seq_len, n_heads, d_q).transpose(1, 0, 2)
    score = Q @ K.transpose(0, 2, 1) / np.sqrt(d_q)
    score = score - np.max(score, axis=-1, keepdims=True)
    score = np.exp(score) / np.sum(np.exp(score), axis=-1, keepdims=True)
    Ot = score @ V # (n_heads, seq_len, d_q)
    Ot = Ot.transpose(1, 0, 2).reshape(seq_len, d_model) # (seq_len, n_heads * d_q)
    return Ot
```

Python

MQA: Multi-Query Attention

```
def multiquery_attention(X: np.ndarray, W_queries: list, W_key: np.ndarray, W_value:
np.ndarray, W_out: np.ndarray) -> np.ndarray:
    """
    X: Input array of shape (seq_len, d_model)
    W_queries: List of query weight matrices, each (d_model, d_k), one per head
    W_key: Shared key weight matrix of shape (d_model, d_k)
    W_value: Shared value weight matrix of shape (d_model, d_v)
    W_out: Output projection matrix of shape (num_heads * d_v, d_model)
    """
    Qs = [X @ W for W in W_queries] # list of (seq_len, d_k)
    num_heads = len(Qs)
    K, V = X @ W_key, X @ W_value # K: (seq_len, d_k); V: (seq_len, d_v)
    Q = np.concatenate(Qs, axis=-1) # (seq_len, d_k * num_q)

    seq_len = Q.shape[0]

    d_k = K.shape[1]
    d_v = V.shape[1]

    Q = Q.reshape(seq_len, num_heads, d_k)
    Q = Q.transpose(1, 0, 2)
    score = Q @ K[None, :, :].transpose(0, 2, 1) / np.sqrt(d_k)
    score = score - np.max(score, axis=-1, keepdims=True)
    score = np.exp(score) / np.sum(np.exp(score), axis=-1, keepdims=True)
    # (num_q, seq_len, seq_len)
    res = score @ V[None, :, :]
    res = res.transpose(1, 0, 2)
    res = res.reshape(seq_len, num_heads * d_v)
    res = res @ W_out

    return res
```

Python

GQA: Grouped Query Attention

```
def grouped_query_attention(Q, K, V, num_heads, num_kv_heads):
    """
    Q: Query tensor, shape (batch_size, seq_len, num_heads * head_dim)
    K: Key tensor, shape (batch_size, seq_len, num_kv_heads * head_dim)
    V: Value tensor, shape (batch_size, seq_len, num_kv_heads * head_dim)
    num_heads: Number of query heads
    num_kv_heads: Number of key/value heads
    """
    num_group = num_heads // num_kv_heads
    batch_size = Q.shape[0]
    seq_len = Q.shape[1]
    head_dim = 0. shape[2] // num_heads
    Q = Q.reshape(batch_size, seq_len, num_heads, head_dim)
    K = K.reshape(batch_size, seq_len, num_kv_heads, head_dim)
    V = V.reshape(batch_size, seq_len, num_kv_heads, head_dim)
    Q = Q.reshape(batch_size, seq_len, num_kv_heads, num_group, head_dim)

    Q = Q.transpose(0, 2, 3, 1, 4)
    K = K.transpose(0, 2, 1, 3)
    V = V.transpose(0, 2, 1, 3)

    d_k = head_dim
    score = Q @ K[:, :, None, :, :].transpose(0, 1, 2, 4, 3) / np.sqrt(d_k)
    score = score - np.max(score, axis=-1, keepdims=True)
    score = np.exp(score) / np.sum(np.exp(score), axis=-1, keepdims=True)
    out = score @ V[:, :, :, None, :]. # (batch_size, num_kv_heads, num_group, seq_len, head_dim)
    out = out.transpose(0, 3, 1, 2, 4)
    out = out.reshape(batch_size, seq_len, num_heads * head_dim)
    return out
```

Python

RoPE (Rotary Position Embedding)

```
def apply_rope(x: np.ndarray, positions: np.ndarray, base: float = 10000.0) -> np.ndarray:
    """
    x: Input embeddings of shape (seq_len, d), d must be even
    positions: Position indices of shape (seq_len,)
    base: Base for frequency computation (default: 10000.0)
    """
    d = x.shape[-1]
    assert d % 2 == 0, "embedding dim must be even"
    i_p = np.arange(d // 2) # (d // 2)
    freq = 1 / np.pow(base, 2 * i_p / d) # (d // 2, )
    theta = positions[:, None] @ freq[None, :] # (seq_len, d // 2)
    A = x[:, 0::2]
    B = x[:, 1::2] # # (seq_len, d // 2)
    cos_theta, sin_theta = np.cos(theta), np.sin(theta) # (seq_len, d // 2)
    A_new = A * cos_theta - B * sin_theta
    B_new = A * sin_theta + B * cos_theta
    out = np.empty_like(x)
    out[:, 0::2] = A_new[:, :]
    out[:, 1::2] = B_new[:, :]
    return out
```

Python

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

SGD Optimizer

```
def sgd(grad, x, lr=0.01, steps=10):
    for _ in range(steps):
        x = x - lr * grad(x)
    return x
```

Adam/AdamW Optimizer

AdamW

```
def adamw(grad, x, lr=1e-3, beta1=0.9, beta2=0.999, eps=1e-8, wd=1e-2, steps=10):
    m, v = 0, 0

    for t in range(1, steps + 1):
        g = grad(x)
        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * g ** 2
        m_hat = m / (1 - beta1 ** t)
        v_hat = v / (1 - beta2 ** t)
        x = x * (1 - lr * wd)
        x = x - lr * m_hat / (np.sqrt(v_hat) + eps)

    return x
```

Weight initialization

Kaiming Init

```
def he_initialization(n_in: int, n_out: int, mode: str = 'fan_in', distribution: str = 'normal',
seed: int = None) -> np.ndarray:
    if seed is not None:
        np.random.seed(seed)
    fan = n_in if mode == "fan_in" else n_out
    if distribution == "normal":
        std = np.sqrt(2/fan)
        return np.random.normal(0.0, std, size=(n_in, n_out))
    else:
        return np.random.uniform(-np.sqrt(6 / fan), np.sqrt(6 / fan), size=(n_in, n_out))
```

Xavier Init

```
def xavier_init(fan_in: int, fan_out: int, mode: str = 'uniform', seed: int = 42) -> dict:
    if seed is not None:
        np.random.seed(seed)
    std = np.sqrt(2 / (fan_out + fan_in)) if mode == 'normal' else np.sqrt(6 / (fan_in + fan_out))
    if mode == 'normal':
        weights = np.random.normal(0.0, std, (fan_in, fan_out))
    elif mode == 'uniform':
        weights = np.random.uniform(-std, std, (fan_in, fan_out))
    else:
        raise ValueError("mode must be 'uniform' or 'normal'")
    return {
        'weights': np.round(weights, 4),
        'shape': (fan_in, fan_out),
        'param': np.round(std, 4)
    }
```

Stage 4: Advanced Architectures

KV cache

```
Q, K, V = x_new @ W_Q, x_new @ W_K, x_new @ W_V
Q = Q[:, None, :]
K = K[:, None, :]
V = V[:, None, :]

if cache is not None:
    K = np.concatenate([cache[0], K], axis=1)
    V = np.concatenate([cache[1], V], axis=1)

updated_cache = (K, V)
score = Q @ K.transpose(0, 2, 1) / np.sqrt(W_Q.shape[1])
score = score - np.max(score, axis=-1, keepdims=True)
score = np.exp(score) / np.sum(np.exp(score), axis=-1, keepdims=True)
output = score @ V
output = np.squeeze(output, axis=1)
return (output, updated_cache)
```

Flash-Attention

```
def flash_attention_forward(Q: np.ndarray, K: np.ndarray, V: np.ndarray,
block_size: int = 2) -> np.ndarray:
    n, d = Q.shape
    0 = np.zeros_like(Q, dtype=float)

    for i in range(0, n, block_size):
        Qb = Q[i:i + block_size]
        num = np.zeros((Qb.shape[0], d)) # (block_size, d)
        den = np.zeros(Qb.shape[0]) # (block_size,)

        for j in range(0, n, block_size):
            Kb = K[j:j + block_size] # (block_size, d)
            Vb = V[j:j + block_size]
            score = Qb @ Kb.T / np.sqrt(d)
            exp_score = np.exp(score) # (block_size, block_size)
            num += exp_score @ Vb # (block_size, d)
            den += exp_score.sum(axis=1)
        0[i:i + block_size] = num / den[:, None]

    return 0
```

Beam Search

```
def beam_search_decode(log_probs: np.ndarray, beam_width: int, top_k: int = 1) -> list:
    beams = [([], 0.0)]
    for step in log_probs:
        token_ids = np.argsort(step)[-beam_width:][::-1]

        candidates = [
            (seq + [tok], score + step[tok])
            for seq, score in beams
            for tok in token_ids
        ]
        beams = sorted(candidates, key=lambda x: x[1], reverse=True)[:beam_width]
    return beams
```

MoE / Top-k routing

```
def moe_top_k(X, router_logits, experts, k):
    B = router_logits.shape[0]

    ids = np.argsort(router_logits, axis=1)[:, -k:] # (B, k)
    sel_logits = router_logits[np.arange(B)[:, None], ids] # (B, k)

    sel_logits = sel_logits - sel_logits.max(axis=1, keepdims=True)
    exp_logits = np.exp(sel_logits)
    weights = exp_logits / exp_logits.sum(axis=1, keepdims=True)

    H = experts[0](X[:,1]).shape[1]
    out = np.zeros((B, H))
    for b in range(B):
        x_b = X[b:b+1] # keep batch dim: (1, d)
        for j in range(k):
            e = ids[b, j] # selected expert id
            w = weights[b, j] # routing weight
            out[b:b+1] += w * experts[e](x_b)
    return out
```

Stage 5: Reinforcement Learning

DPO (Direct Preference Optimization)

Core idea: Optimize policy directly from preference data without need for explicit reward model.

$$\mathcal{L}_{DPO}(\pi_{\theta}; \pi_{ref}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

```
def dpo_loss(log_w, log_l, ref_log_w, ref_log_l, beta=0.5):
    policy_ratio = log_w - log_l
    ref_ratio = ref_log_w - ref_log_l
    dpo_logits = beta * (policy_ratio - ref_ratio)
    loss = -np.log(1.0 / (1.0 + np.exp(-dpo_logits))) + 1e-8
    return loss.mean()
```

PPO (Proximal Policy Optimization)

Core idea: Optimize policy with clipped advantage to prevent divergence from old policy.

$$L^{CLIP}(\theta) = \mathbb{E}_{\tau} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

* $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{old}(a_t|s_t)}$: probability ratio

GAE (Generalized Advantage Estimation):

$$\hat{A}_t = \sum_{k=0}^{T-t} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

* δ_t : TD error at step t

* γ : discount factor, λ : GAE smoothing ($\lambda = 1 \rightarrow \text{MC}$, $\lambda = 0 \rightarrow \text{TD}$)

```
def compute_gae(rewards, values, gamma=0.99, lam=0.95):
    # rewards, values: (T,). values has one extra bootstrap value at end
    advantages = np.zeros(len(rewards))
    gae = 0
    for t in reversed(range(len(rewards))):
        delta = rewards[t] + gamma * values[t + 1] - values[t]
        gae = delta + gamma * lam * gae
        advantages[t] = gae
    return advantages

def ppo_loss(log_probs, old_log_probs, advantages, epsilon=0.2):
    ratio = np.exp(log_probs - old_log_probs)
    clipped_ratio = np.clip(ratio, 1 - epsilon, 1 + epsilon)
    surr_loss = -np.minimum(ratio * advantages, clipped_ratio * advantages)
    return surr_loss.mean()
```

Full PPO objective (with value loss and entropy):

```
def ppo_total_loss(log_probs, old_log_probs, advantages, values, targets,
entropy, c1=0.5, c2=0.01, eps=0.2):
    policy_loss = ppo_loss(log_probs, old_log_probs, advantages, eps)
    value_loss = c1 * ((values - targets) ** 2).mean()
    entropy_bonus = -c2 * entropy.mean()
    return policy_loss + value_loss + entropy_bonus
```

GRPO (Group Relative Policy Optimization)

$$\mathcal{L}_{GRPO}(\theta) = -\mathbb{E} \left[\sum_{i=1}^G \min \left(r_i(\theta) \hat{A}_i, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_{\theta} \| \pi_{old}) \right]$$

$$\hat{A}_i = \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G})}$$

```
def grpo_loss(log_probs, old_log_probs, ref_log_probs, rewards, epsilon=0.2, beta=0.01):
    # rewards: (B, G) - B prompts, G rollouts each
    # log_probs, old/ref_log_probs: (B*G, T) - per-token log probs (gradient flows per token)
    advantages = (rewards - rewards.mean(axis=1, keepdims=True)) / (rewards.std(axis=1, keepdims=True) + 1e-8)
    advantages = advantages.reshape(-1)[:, None] # (B*G, 1) - response-level, broadcasts over T

    ratio = np.exp(log_probs - old_log_probs) # (B*G, T) - token-level ratio
    clipped = np.clip(ratio, 1 - epsilon, 1 + epsilon)
    policy_loss = -np.minimum(ratio * advantages, clipped * advantages).mean()

    # KL penalty (estimator): (n_ref / n_0) - log(n_ref / n_0) - 1
    r_kl = np.exp(ref_log_probs - log_probs) # (B*G, T)
    kl = r_kl - np.log(r_kl) - 1
    return policy_loss + beta * kl.mean()
```

Stage 6 Training Loop Coding

MLP (SwiGLU FFN)

```
class SwiGLU_FFNN(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.up_gate = nn.Linear(d_in, 2 * d_hidden, bias=False)
        self.down = nn.Linear(d_hidden, d_out, bias=False)
        self.silu = nn.SiLU()

    def forward(self, x):
        gate, up = self.up_gate(x).chunk(2, dim=-1)
        return self.down(self.silu(gate) * up)
```

Transformer Block

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff):
        super().__init__()
        self.attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.ln1 = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
        )
        self.ln2 = nn.LayerNorm(d_model)

    def forward(self, x, attn_mask=None):
        h, _ = self.attn(x, x, x, attn_mask=attn_mask)
        x = self.ln1(x + h)
        h = self.mlp(x)
        x = self.ln2(x + h)
        return x
```

PyTorch Training Loop - Full

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"

X = torch.randn(1000, 20)
true_W = torch.randn(20, 3)
y = (X @ true_W).argmax(dim=1)

loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(20, 64),
            nn.LayerNorm(64),
            nn.ReLU(),
            nn.Dropout(p=0.1),
            nn.Linear(64, 3)
        )

    def forward(self, x):
        return self.net(x)

model = MLP().to(device)
loss_fn = nn.CrossEntropyLoss()
optim = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-2)
```

```
# Warmup + cosine decay (stepped per batch)
num_epochs = 10
num_steps = num_epochs * len(loader)
warmup_steps = int(0.1 * num_steps) # 10% of total steps

warmup = torch.optim.lr_scheduler.LinearLR(
    optim, start_factor=1e-8, end_factor=1.0, total_iters=warmup_steps
)
cosine = torch.optim.lr_scheduler.CosineAnnealingLR(
    optim, T_max=num_steps - warmup_steps
)
scheduler = torch.optim.lr_scheduler.SequentialLR(
    optim, schedulers=[warmup, cosine], milestones=[warmup_steps]
)
```

```
for epoch in range(num_epochs):
    model.train()
    total_loss = 0

    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)

        logits = model(xb)
        loss = loss_fn(logits, yb)

        optim.zero_grad()
        loss.backward()
        optim.step()
        scheduler.step() # advance LR every batch

        total_loss += loss.item()

    model.eval()
    with torch.no_grad():
        logits = model(X.to(device))
        eval_loss = loss_fn(logits, y.to(device)).item()
        acc = (logits.argmax(dim=1) == y.to(device)).float().mean().item()

    print('Epoch ', epoch, 'loss: ', total_loss / len(loader), 'eval_loss: ',
eval_loss, 'acc: ', acc)
```

LLM Model Definition

```
class TinyLLM(nn.Module):
    def __init__(self, vocab_size, d_model=128, n_heads=4, d_ff=512, n_layers=2, max_len=256):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_len, d_model)
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_heads, d_ff) for _ in range(n_layers)
        ])
        self.ln_f = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size)

    def forward(self, input_ids, labels=None):
        B, T = input_ids.shape
        pos = torch.arange(T, device=input_ids.device)

        x = self.token_emb(input_ids) + self.pos_emb(pos)[None, :, :]

        for block in self.blocks:
            x = block(x)

        x = self.ln_f(x)
        logits = self.lm_head(x) # (B, T, vocab_size)
        loss = None
        if labels is not None:
            loss = F.cross_entropy(
                logits[:, :-1, :], labels[:, :-1, :].reshape(-1, logits.size(-1)),
                labels[:, 1:].reshape(-1)
            )
        return logits, loss
```

High-Level Torch Functions

```
import torch
import torch.nn as nn
import torch.nn.functional as F
## Layers
nn.Linear(in_features, out_features, bias=True)
nn.Embedding(num_embeddings, embedding_dim)
nn.LayerNorm(normalized_shape)
nn.RMSNorm(normalized_shape)
nn.Dropout(p=0.5)
nn.BatchNorm1d(num_features)
nn.MultiheadAttention(embed_dim, num_heads, batch_first=True)
## Activations
nn.ReLU()
nn.GELU()
nn.SiLU()
nn.Sigmoid()
nn.Tanh()
nn.Sequential(*layers)
## Losses
nn.CrossEntropyLoss()
nn.MSELoss()
nn.BCEWithLogitsLoss()
F.cross_entropy(logits, labels)
F.mse_loss(pred, target)
F.binary_cross_entropy_with_logits(logits, labels)
torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
torch.optim.Adam(model.parameters(), lr=1e-3)
torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=0.01)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```